RAJALAKSHMI ENGINEERING COLLEGE

Department of Cyber Security

2026-2027 - Odd Semester

III Year / V Sem

CS23533 - FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Review 1 – 23.09.2026

MITRE ATTACK Rule-Based Navigator

Batch Members: Pranathee H (241901078), Sivaranjanii M (241901109)

Supervisor: Dr. S. Bhuvana, Assistant Professor [SG]

Introduction

- MITRE ATTACK is a public knowledge base of adversary techniques used by security teams worldwide
- Analysts see raw system behaviours (process creation, registry writes, failed logins) and must map them to techniques by hand
- This project builds a rule-based expert system that does that mapping automatically and recommends ATTACK mitigations
- Uses classical AI only: production rules, forward chaining, best-first search. No machine learning

Problem Statement

- Mapping observed behaviours to ATTACK techniques is manual and depends on analyst experience
- Alerts arrive faster than they can be triaged, and partial evidence is easy to miss
- Machine-learning detectors output a score with no reasoning an analyst can audit
- Nothing links a detected technique back to its recommended mitigation automatically

Motivation

- ATTACK is well documented but not directly executable as detection logic
- Explainability matters in security: an analyst must be able to see why a technique was flagged
- A rule chain gives an audit trail; a statistical model does not
- The knowledge base is editable, so new techniques can be added without retraining anything

Objectives

- To build a rule-based expert system that maps observed behaviours to likely ATTACK techniques
- To use a goal-based intelligent agent with IF-THEN rules to control the reasoning
- To apply forward chaining to infer the technique from the observed facts
- To apply best-first search to rank the rules by how strongly the evidence supports them
- To output the risk level, mitigation steps and a reasoning trace for each technique

Literature Survey

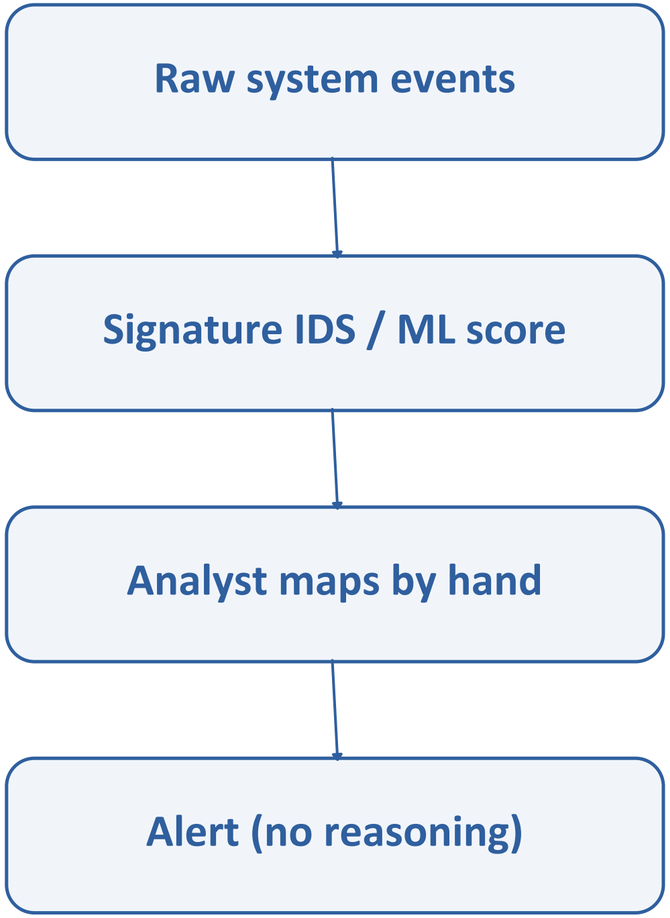
S.No.	Author & Year	Method / Technique	Key Findings	Limitations
1	B. E. Strom et al. (2018)	Knowledge base of adversary tactics and techniques built from real-world behaviour	A common language for describing attacker behaviour	A catalogue only; does not reason over live observations
2	U. Lindqvist & P. A. Porras (1999)	Forward-chaining production rules (P-BEST expert system)	Rules can detect misuse from observed events	Not tied to ATT&CK; no mitigation advice; no rule ranking
3	G. Husari et al. (2017)	Ontology and text processing to map threat reports to ATT&CK (TTPDrill)	Threat actions can be extracted from written reports	Works on reports after an incident, not on live behaviour

Literature Survey (contd.)

S.No.	Author & Year	Method / Technique	Key Findings	Limitations
4	P. E. Hart, N. J. Nilsson & B. Raphael (1968)	Heuristic best-first search with a priority queue (A*)	A heuristic finds the best candidate first	General purpose; needs a security-specific heuristic
5	C. L. Forgy (1982)	Rete algorithm for fast matching in rule-based systems	Makes matching many rules against many facts efficient	Speeds up matching only; gives no security knowledge

Existing System

- Approach: signature-based IDS and antivirus, ML detectors, and analysts mapping activity to ATTACK by hand
- Advantages: fast on known threats; ML can spot unusual patterns
- Limitations: misses behaviour-based attacks; ML gives no reason for its decision; mapping is slow; no built-in mitigation

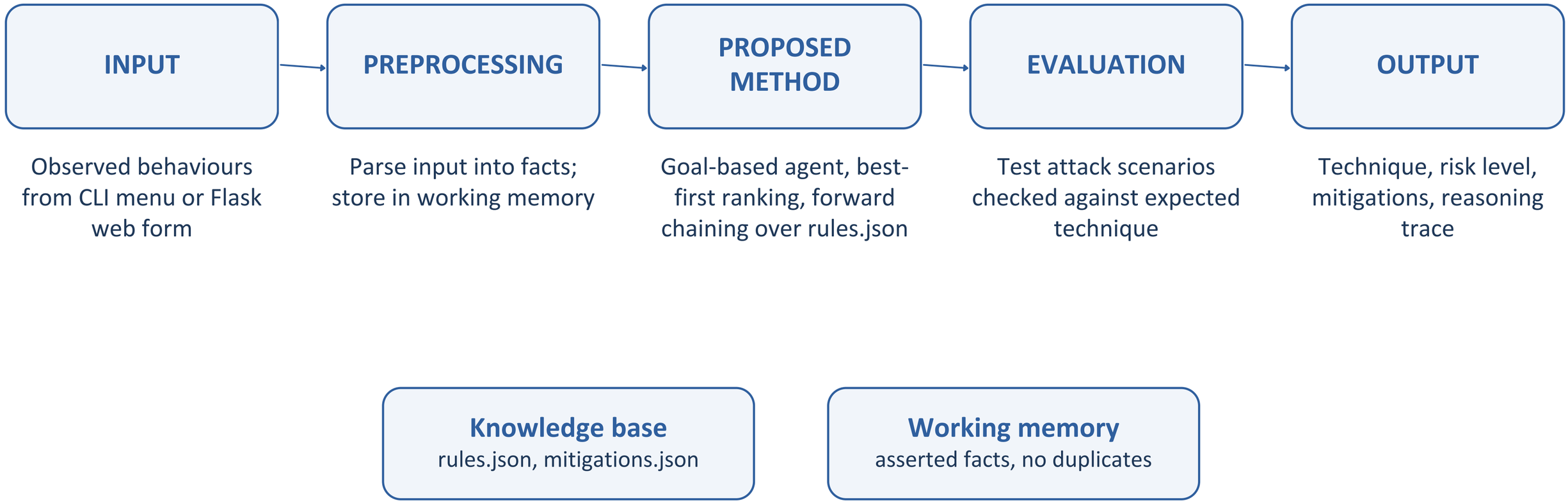


Proposed System

- A Python goal-based agent that perceives observed behaviours and reasons until a technique is matched
- IF-THEN rules, forward chaining and best-first search over an editable knowledge base
- Improves the existing system: explainable, ranks by evidence strength, includes mitigations
- Expected outcome: a ranked technique list with risk level, mitigations and trace



System Architecture / Block Diagram



Methodology

1. Input
Take observed behaviours from the CLI or web form

2. Preprocessing
Parse the input into facts

3. Matching
Match facts against the IF-THEN rules

4. Ranking
Best-first search scores each rule: matched ÷ total conditions

5. Evaluation
Fire full matches; check test scenarios against expected output

6. Output
Report technique, risk, mitigations and trace

Technologies Used

- Programming Language: Python 3.10+
- Framework / Library: heapq, json, dataclasses, collections (standard library); Flask for the web interface
- Database: none needed; knowledge stored in JSON files (rules.json, mitigations.json)
- Tools / IDE: VS Code, Git and GitHub
- Hardware: Intel Core i3 or higher, 4 GB RAM, no GPU

Module Description

Module 1 – Knowledge Base & Facts
Rule format, rules.json, mitigations.json

Module 2 – Forward Chaining Engine
Working memory, matching, firing rules, explanation trace

Module 3 – Best-First Search
Heuristic scoring and priority queue to rank rules

Module 4 – Agent & Interface
Goal-based agent, input parser, CLI menu, report format

Module 5 – Flask Web Interface
Web front end over the same agent and engine

Preliminary / Expected Results

Expected detections

Scenario	Observed behaviour	Expected technique
PowerShell intrusion	powershell.exe spawned, encoded command	T1059.001 PowerShell
Registry persistence	Run key modified, new startup entry	T1547 Registry Run Keys
Brute force	More than 5 failed logins from one source	T1110 Brute Force

What we expect

- Ranked list of techniques, best-supported first
- Partial matches shown as "possible" techniques
- Every result has a risk level, mitigations and a reasoning trace

Project Schedule

Activity	Month 1	Month 2	Month 3	Month 4
Literature Survey	✓			
Problem Identification	✓			
System Design		✓		
Implementation		✓	✓	
Testing & Evaluation			✓	✓

Current Progress

- ✓ Literature survey completed
- ✓ Problem identification completed
- ✓ Requirement analysis / methodology finalized
- □ Implementation completed
- □ Testing completed
- □ Final evaluation completed

Future Work

- Complete the implementation of all modules
- Test with attack scenarios such as PowerShell, registry persistence and brute force
- Compare with plain sequential rule scanning
- Measure detection accuracy and ranking quality
- Add the Flask web interface; prepare the final report and documentation

Conclusion

- A rule-based expert system will map observed behaviours to MITRE ATTACK techniques using classical AI only
- Novelty: runtime mapping from behaviours, explainable decisions, best-first ranking and built-in mitigations
- Expected outcome: an offline, lightweight tool that helps analysts triage faster and understand every decision

References

[1] B. E. Strom et al., “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, 2018 (rev. 2020).
[2] U. Lindqvist and P. A. Porras, “Detecting Computer and Network Misuse Through the Production-Based Expert System Toolset (P-BEST),” Proc. IEEE Symp. Security and Privacy, 1999.
[3] G. Husari et al., “TTPDrill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources,” Proc. ACSAC, 2017.
[4] P. E. Hart, N. J. Nilsson and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Trans. Systems Science and Cybernetics, 1968.
[5] C. L. Forgy, “Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem,” Artificial Intelligence, 1982.